

Módulo 8 – Capítulo 06 – Sección 01

Por qué hacer fine-tuning: casos donde el prompting no es suficiente

El fine-tuning es la técnica de mayor costo de ingeniería en el flujo de trabajo de modelos locales, y por esa razón debe ser la última opción en la secuencia de decisiones, no la primera. La secuencia correcta es: primero, prompt engineering con el modelo base; segundo, few-shot prompting con ejemplos en el contexto; tercero, RAG para inyectar conocimiento externo; y solo cuando todos estos han demostrado empíricamente alcanzar un plateau de rendimiento insuficiente, fine-tuning. Esta secuencia no es dogma sino economía: cada paso previo al fine-tuning es más rápido de implementar, más fácil de iterar y menos costoso de operar.

¿Qué evidencia indica que el prompting ha alcanzado su límite? La señal más clara es una curva de aprendizaje flat en el golden dataset: añadir más ejemplos al few-shot prompt ya no mejora el score de calidad en el golden set. La segunda señal es inconsistencia estructural: el modelo produce la respuesta correcta en el 85-90% de los casos pero falla en el 10-15% restante de una forma que no puede corregirse con instrucciones adicionales en el prompt. La tercera señal es el costo del contexto: cuando el few-shot prompting requiere 1.000-2.000 tokens de ejemplos en cada petición, el costo de tokens de entrada en producción puede superar el presupuesto proyectado, y un modelo fine-tuneado que "sabe" el comportamiento esperado sin necesitar esos ejemplos en cada petición es más económico.

Los escenarios concretos donde el fine-tuning aporta valor medible incluyen formatos de salida altamente específicos que el few-shot prompting no produce con consistencia suficiente para producción. Por ejemplo, un sistema de extracción de información médica que debe producir JSON con un schema de 40 campos específicos del dominio clínico: con few-shot prompting sobre un modelo general de 7B, la tasa de conformidad al schema puede estar entre el 82-88%; con fine-tuning sobre 2.000 ejemplos anotados, puede elevarse al 97-99%. Esta diferencia de tasa de error del 2-15% puede significar la diferencia entre un sistema usable y uno que requiere revisión manual constante.

El conocimiento de dominio muy especializado es otro escenario clásico: terminología médica interna de un hospital, el lenguaje de APIs propietarias de una empresa, los estilos de redacción de un medio de comunicación específico. Un modelo general "inventa" términos cuando no conoce la terminología exacta o produce respuestas plausibles pero incorrectas en el dominio especializado. El fine-tuning sobre documentación real del dominio internaliza ese vocabulario en los pesos del modelo, eliminando las alucinaciones de terminología que el prompting no puede resolver.

La eficiencia en inferencia es el caso de uso económico más convincente: un modelo de 7B fine-tuneado en una tarea específica puede igualar o superar a un modelo de 70B con few-shot prompting en esa tarea, con un costo de inferencia 10x menor. Para aplicaciones de alto volumen (millones de peticiones al mes), esta diferencia de costo puede financiar varios ciclos de fine-tuning con los ahorros generados en el primer mes de producción.

Casos de uso donde el fine-tuning supera al prompting

- **Formato de salida estricto y consistente:** cuando se requiere que >98% de las respuestas sigan un schema JSON o un formato específico que el few-shot produce con 85-90% de consistencia; el fine-tuning puede elevar esto al 97-99%.
- **Dominio con terminología no en los datos de preentrenamiento:** sistemas legacy, APIs propietarias, jerga interna de la empresa; el modelo base "inventa" terminología; el fine-tuning sobre documentación real resuelve el problema.
- **Eficiencia en inferencia:** un modelo de 7B fine-tuneado puede igualar a un modelo de 70B con few-shot en la tarea específica; el costo de inferencia 10x menor justifica el costo único de fine-tuning.
- **Reducción de longitud del prompt:** una capacidad que requiere 500-1.000 tokens de contexto en few-shot puede reducirse a un system prompt de 50 tokens post fine-tuning, multiplicando el contexto efectivo disponible para datos de usuario.
- **Comportamiento ante edge cases:** el few-shot no garantiza comportamiento correcto en inputs que difieren del patrón de los ejemplos; el fine-tuning sobre un dataset diverso mejora la robustez ante entradas inesperadas.

Nota del Arquitecto: La pregunta que siempre hago antes de aprobar un proyecto de fine-tuning es: "¿Has medido el rendimiento de GPT-4o con few-shot prompting en el golden dataset?" Si la respuesta es no, no estamos listos para fine-tuning. El modelo de frontera con few-shot es el upper bound de calidad alcanzable con

prompting; si ese upper bound ya supera el umbral del producto, el camino más rápido es fine-tuning de un modelo más pequeño. Si no lo supera, el problema está en el diseño de la tarea o los datos, no en la técnica de adaptación.

El fine-tuning es la herramienta correcta en los escenarios identificados. Las secciones siguientes presentan las técnicas específicas de fine-tuning eficiente —LoRA, QLoRA y DPO— que permiten ejecutar este proceso en hardware de consumo sin los recursos de un laboratorio de investigación.

Módulo 8 – Capítulo 06 – Sección 02

LoRA (Low-Rank Adaptation): adaptar modelos grandes con parámetros mínimos

El fine-tuning completo de un modelo de 7B parámetros en BF16 requiere calcular gradientes para todos los 7.000 millones de parámetros, almacenar el estado del optimizador Adam (dos momentos por parámetro: $2 \times 7B \times 4 \text{ bytes} = 56 \text{ GB}$ solo para el optimizador), y los buffers de gradiente (otros 14 GB). El total supera los 100 GB de VRAM para un modelo de 7B, lo que hace el full fine-tuning impracticable en cualquier hardware que no sea un sistema multi-GPU de datacenter de alta gama. LoRA resuelve este problema con una observación matemática elegante: las actualizaciones de pesos durante el fine-tuning son intrínsecamente de bajo rango.

La idea central de LoRA es que para una capa de peso $W_0 \in \mathbb{R}^{(d \times k)}$, en lugar de aprender la actualización completa $\Delta W \in \mathbb{R}^{(d \times k)}$, se aprende una aproximación de bajo rango $\Delta W = BA$ donde $B \in \mathbb{R}^{(d \times r)}$ y $A \in \mathbb{R}^{(r \times k)}$ con $r \ll \min(d, k)$. La actualización efectiva es $\tilde{W} = W_0 + \alpha BA/r$ donde α es el hiperparámetro de escala (lora_alpha). Para un modelo Llama 3 8B con rank $r=16$, los adaptadores LoRA en las proyecciones Q, K, V y O de todas las 32 capas del transformer suman aproximadamente 41 millones de parámetros entrenables, comparado con los 8.000 millones del modelo completo. El ratio de compresión es de casi 200x en parámetros entrenables.

Esta reducción tiene consecuencias prácticas inmediatas: los gradientes y el estado del optimizador solo se calculan para los 41 millones de parámetros LoRA, no para los 8.000 millones congelados del modelo base. El resultado es que el fine-tuning LoRA de Llama 3 8B en BF16 completo requiere aproximadamente 20 GB de VRAM —dentro del alcance de GPUs de 24 GB de consumo— mientras el full fine-tuning requeriría más de 100 GB. Cuando se combina con la cuantización del modelo base a 4 bits (QLoRA, presentada en la siguiente sección), el requisito de VRAM cae a 8-10 GB, haciendo el fine-tuning viable en GPUs de consumo de 12 GB o incluso 8 GB.

En inferencia, los adaptadores LoRA no introducen overhead de latencia cuando se fusionan con el modelo base: `model.merge_and_unload()` de la librería PEFT calcula $W = W_0 + \alpha BA/r$ para cada capa y almacena el resultado directamente en los pesos del modelo, produciendo un modelo fusionado idéntico en arquitectura al modelo base original pero con los pesos ajustados al dominio de fine-tuning. El modelo fusionado puede ser exportado a GGUF para servicio con Ollama/llama.cpp o desplegado directamente en vLLM sin ningún overhead adicional respecto al modelo base sin adaptadores.

Los hiperparámetros de LoRA más importantes son: `r` (el rango de la aproximación, típicamente 8-64), `lora_alpha` (el factor de escala, frecuentemente igual a `r` o el doble), y `target_modules` (qué proyecciones reciben adaptadores). Para tareas de conocimiento de dominio o formato específico, `target_modules=["q_proj", "k_proj", "v_proj", "o_proj"]` es el estándar; incluir las capas MLP (`gate_proj`, `up_proj`, `down_proj`) mejora la calidad en tareas de conocimiento pero duplica el número de parámetros LoRA. La variante DoRA (Weight-Decomposed Low-Rank Adaptation) separa la actualización en magnitud y dirección, consistentemente superando a LoRA estándar con el mismo número de parámetros: disponible como `use_dora=True` en la librería PEFT.

Conceptos técnicos de LoRA

- **Módulos objetivo típicos:** `q_proj`, `k_proj`, `v_proj`, `o_proj` en las capas de atención; incluir capas MLP mejora calidad en tareas de conocimiento pero duplica parámetros.
- **Selección de rank:** `r=16` es el punto de inicio estándar; `r=4-8` para tareas simples (clasificación, reformato); `r=32-64` para tareas complejas de razonamiento o conocimiento profundo.
- **DoRA:** variante que descompone la actualización en magnitud y dirección; consistentemente supera a LoRA estándar; disponible como `use_dora=True` en PEFT.
- **Saving y carga:** los adaptadores se guardan como archivos de ~50-300 MB en formato SafeTensors; se cargan y fusionan en segundos con `PeftModel.from_pretrained()`.
- **Stacking de adaptadores:** múltiples adaptadores LoRA para distintas tareas; cambio dinámico con `model.set_adapter("tarea_1")`; útil para servir múltiples especializaciones del mismo modelo base.

Nota del Arquitecto: El rango del adaptador LoRA es frecuentemente el hiperparámetro más importante para la calidad del fine-tuning. Con `r=8` y datos de

alta calidad, obtendrás buenos resultados para tareas de formato y estilo. Con $r=16$, cubrirás la mayoría de las tareas de conocimiento de dominio. Solo en casos donde el dominio es extremadamente técnico y los ejemplos son pocos necesitarás $r=32-64$. Escalar el rango más allá de ese punto pocas veces mejora la calidad y aumenta el riesgo de overfitting.

LoRA establece la base de todos los métodos de fine-tuning eficiente presentados en este capítulo. La sección siguiente presenta QLoRA, que combina LoRA con cuantización del modelo base a 4 bits para hacer viable el fine-tuning en hardware de consumo.

Módulo 8 – Capítulo 06 – Sección 03

QLoRA: fine-tuning con cuantización de 4 bits para hardware limitado

LoRA reduce el número de parámetros entrenables en 200x, pero el modelo base congelado — cuyos pesos se mantienen fijos durante el entrenamiento— sigue siendo el mayor componente de memoria. Un modelo Llama 3 8B en BF16 ocupa 16 GB solo en pesos base. QLoRA resuelve este componente aplicando cuantización a 4 bits al modelo base antes de entrenar, reduciendo a la mitad los 16 GB de pesos base a ~8 GB, lo que combinado con el mínimo de los adaptadores LoRA y el optimizador hace viable el fine-tuning de modelos de 7B en GPUs de 8-12 GB de VRAM.

La innovación central de QLoRA no es simplemente aplicar INT4 al modelo base. El tipo de dato NF4 (NormalFloat 4-bit), diseñado específicamente para los valores de los pesos de redes neuronales entrenadas con SGD, es la contribución que separa QLoRA de la cuantización naive a 4 bits. Los valores de los pesos de una red neuronal entrenada siguen aproximadamente una distribución normal estándar: la mayoría de los valores están concentrados cerca de cero, con colas que decaen hacia ± 3 o ± 4 desviaciones estándar. INT4 uniforme distribuye sus 16 posibles valores de forma equidistante en ese rango, lo que implica que pasa muchos "slots" de cuantización en regiones de baja densidad de pesos y pocos en la región de alta densidad alrededor de cero. NF4, en cambio, posiciona sus 16 valores en los cuantiles de la distribución normal estándar: más valores posibles cerca de cero donde están la mayoría de los pesos, y menos en las colas. El resultado es un error de cuantización un 35% menor que INT4 uniforme para los pesos típicos de una red neuronal.

La double quantization de QLoRA lleva la compresión un nivel más: los factores de escala de la cuantización NF4 (que en cuantización por grupos de 64 son un tensor de escala almacenado en FP32, uno por grupo de 64 pesos) se cuantizan a su vez en FP8, reduciendo el uso de memoria de los factores de escala de 4 bytes a 1 byte por elemento. El ahorro es ~0.37 bits adicionales por parámetro del modelo base, llevando la compresión total de 4 bits nominales a ~4.5 bits efectivos promedio. La penalización de velocidad de QLoRA respecto a LoRA en BF16 completo es del 15-30% en throughput de entrenamiento: el proceso de

dequantización de NF4 a BF16 que ocurre en cada forward pass tiene un costo computacional real que no existe cuando el modelo base ya está en BF16.

La configuración mínima de QLoRA con la librería PEFT de Hugging Face requiere tres componentes encadenados. Primero, la configuración de cuantización:

```
BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_quant_type="nf4",
bnb_4bit_compute_dtype=torch.bfloat16, bnb_4bit_use_double_quant=True)
```

Segundo, la carga del modelo con esa configuración: `AutoModelForCausalLM.from_pretrained(model_id, quantization_config=bnb_config)`. Tercero, la configuración del adaptador LoRA:

```
LoraConfig(r=16, lora_alpha=32, target_modules=["q_proj", "v_proj"],
task_type="CAUSAL_LM")
```

La combinación de los tres mediante `get_peft_model(model, lora_config)` produce un modelo QLoRA listo para entrenamiento con 8-10 GB de VRAM para un modelo de 7B.

El gradient checkpointing es una optimización complementaria que reduce la memoria de activaciones durante el backward pass recomputando activaciones en lugar de almacenarlas: activando

```
model.enable_input_require_grads() y
training_args.gradient_checkpointing=True
```

en el `TrainingArguments`, el uso de VRAM de activaciones cae de $O(n)$ donde n es el número de capas al costo de recalcularlas durante el backward pass, con un overhead de velocidad del 10-20% que frecuentemente vale la pena para aumentar el batch size o la longitud de secuencia dentro del presupuesto de VRAM.

Aspectos técnicos de QLoRA

- **NF4 (NormalFloat 4-bit):** 16 valores posicionados en cuantiles de distribución normal; reduce error de cuantización 35% vs INT4 para pesos de redes neuronales entrenadas con SGD.
- **Double quantization:** factores de escala de NF4 cuantizados en FP8; ahorro adicional de ~0.37 bits por parámetro; total ~4.5 bits efectivos vs 4 bits nominales.
- **Paginación a CPU (paged optimizers):** `paged_adamw_8bit` en bitsandbytes pagina el estado del optimizador a CPU RAM cuando la VRAM se satura; overhead <5% cuando ocurre raramente.
- **Gradient checkpointing:** recomputa activaciones en lugar de almacenarlas; reduce VRAM de activaciones con overhead de velocidad del 10-20%; activar con `training_args.gradient_checkpointing=True`.
- **Comparación de memoria:** LoRA de 7B en BF16: ~20 GB VRAM; QLoRA de 7B con NF4: ~8-10 GB VRAM; permite fine-tuning en GPUs de consumo de 12 GB.

Nota del Arquitecto: QLoRA democratiza el fine-tuning de modelos de 7B-13B en hardware accesible, pero la penalización del 20-30% en velocidad de entrenamiento importa cuando tienes un dataset grande. Para datasets de más de 50.000 ejemplos o cuando el tiempo de entrenamiento supera las 8 horas, evalúa si acceder a una GPU más grande en la nube (una A100 de 40 GB para LoRA en BF16 sin QLoRA) es más económico que el tiempo adicional de entrenamiento en hardware de consumo.

QLoRA extiende el acceso al fine-tuning a cualquier equipo con hardware de consumo disponible. La sección siguiente presenta los datasets de entrenamiento: la calidad de los datos es el factor más determinante del resultado, independientemente de si se usa LoRA o QLoRA.

Módulo 8 – Capítulo 06 – Sección 04

Datasets para fine-tuning: preparación, formato y calidad de los datos de entrenamiento

Si tuvieras que elegir entre invertir tu tiempo disponible en optimizar hiperparámetros de LoRA o en mejorar la calidad de tus datos de entrenamiento, la respuesta correcta en el 90% de los proyectos de fine-tuning es mejorar los datos. Un dataset de 1.000 ejemplos de alta calidad, bien formateados, representativos de la distribución real de casos de uso, produce invariablemente mejores resultados que 100.000 ejemplos ruidosos, mal formateados o no representativos. La razón es directa: el fine-tuning de instruction following aprende de los patrones en los datos; datos de calidad enseñan los patrones correctos con señal fuerte, mientras datos ruidosos enseñan patrones mezclados con ruido que el modelo debe separar — tarea para la que no está optimizada la función de loss estándar.

El formato más universalmente soportado por las herramientas modernas de fine-tuning es la lista de mensajes con roles `system`, `user` y `assistant`, serializada mediante el chat template específico del modelo base. El chat template es una plantilla Jinja2 definida en el `tokenizer_config.json` del modelo que especifica exactamente cómo serializar la conversación en el string de texto que el tokenizador procesa. Para Llama 3, la serialización de un mensaje de usuario incluye los tokens especiales `<|begin_of_text|>` `<|start_header_id|>user<|end_header_id|>\n\n{contenido}<|eot_id|>`. Aplicar el chat template correcto es crítico: usar el template equivocado es un error silencioso —el entrenamiento completa sin errores pero el modelo aprende a producir tokens de control en lugar del texto correcto, o no aprende el comportamiento instruction-following esperado.

La operación más importante del proceso de preparación de datos que frecuentemente se subestima es el loss masking: durante el entrenamiento de instruction following, la función de loss debe calcularse solo sobre los tokens que corresponden a la respuesta del `assistant`, no sobre los tokens del `system` prompt ni del `user`. Si se calcula loss sobre todos los tokens, el modelo aprende a predecir tanto las preguntas del usuario como las respuestas del asistente, lo que introduce gradientes erróneos que degradan la calidad del instruction following. Las herramientas modernas como TRL con `DataCollatorForCompletionOnlyLM` y Axolotl

implementan este masking automáticamente identificando los tokens de inicio de la respuesta del assistant mediante los tokens especiales del chat template.

Las fuentes de datos para fine-tuning tienen jerarquía de calidad clara. Los datos reales de producción anonimizados son la mejor fuente disponible cuando existen: representan exactamente la distribución de casos de uso reales y los patrones de error que el modelo debe manejar. Los datos sintéticos generados con modelos más capaces (GPT-4o, Claude Opus) usando técnicas como Self-Instruct o Evol-Instruct son una alternativa de alta calidad cuando no hay datos de producción: un modelo de frontera puede generar miles de ejemplos representativos del dominio objetivo con alta consistencia de formato y corrección factual. Los datasets públicos de Hugging Face (Alpaca, OpenHermes, SlimOrca, Capybara) son útiles para preservar las capacidades generales del modelo durante el fine-tuning de dominio específico, mezclados en un ratio del 20% del dataset total para evitar la pérdida de instruction following general.

La deduplicación y limpieza de datos son etapas que toman más tiempo que el entrenamiento mismo pero son determinantes de su calidad. Datasets de producción frecuentemente tienen 30-60% de duplicados que inflan artificialmente el tamaño del dataset y sesgan el entrenamiento hacia los patrones más frecuentes. La deduplicación exacta por SHA-256 del texto normalizado elimina los duplicados perfectos; la deduplicación near-exact con MinHash (disponible en `datasketch.MinHashLSH` con umbral Jaccard de 0.8) elimina las variantes que difieren solo en espaciado o puntuación.

Aspectos técnicos de la preparación de datos

- **Formato ShareGPT vs Alpaca:** ShareGPT usa lista de dicts con `role / content`; Alpaca usa campos planos; Axolotl y LLaMA-Factory convierten ambos al chat template automáticamente.
- **Chat template application:** `tokenizer.apply_chat_template(messages, tokenize=False, add_generation_prompt=False)` serializa la conversación; `add_generation_prompt=True` solo para inferencia.
- **Loss masking:** la loss debe calcularse solo sobre tokens del `assistant`; TRL con `DataCollatorForCompletionOnlyLM` y Axolotl lo implementan automáticamente.
- **Deduplicación:** `datasketch.MinHashLSH` con Jaccard > 0.8 para near-deduplication; SHA-256 del texto normalizado para duplicados exactos; datasets de producción tienen frecuentemente 30-60% de duplicados.

- **Validación de calidad:** muestra aleatoria de 100-200 ejemplos revisada manualmente; histograma de longitudes; clustering de temas con embeddings; verificación de formato correcto en todos los ejemplos.

Nota del Arquitecto: La prueba de fuego de un dataset de fine-tuning es simple: toma los 10 ejemplos más difíciles del golden dataset del Capítulo 1 y verifica que están representados (o que hay ejemplos similares) en el dataset de entrenamiento. Si el dataset no cubre los casos más difíciles del golden set, el fine-tuning no mejorará el rendimiento exactamente donde más importa. Construir el dataset con los casos difíciles representados explícitamente es la decisión de ingeniería de datos más impactante del proyecto.

La calidad del dataset determina el techo de calidad alcanzable con fine-tuning. La sección siguiente presenta las herramientas que ejecutan ese entrenamiento con la mayor eficiencia posible.

Módulo 8 – Capítulo 06 – Sección 05

Herramientas: Unsloth, Axolotl, LLaMA-Factory y Hugging Face TRL

El ecosistema de herramientas de fine-tuning de LLMs se ha consolidado alrededor de cuatro opciones principales con perfiles de uso distintos. La elección de herramienta tiene impacto real en la velocidad de entrenamiento, la flexibilidad de configuración y la barrera de entrada del equipo. Las cuatro no son intercambiables: cada una tiene su caso de uso óptimo.

Unsloth es la herramienta de máxima velocidad bruta. Su equipo ha reimplementado en Triton los kernels críticos del pipeline de entrenamiento con LoRA: los kernels de RoPE embedding, la función de pérdida cross-entropy y el mecanismo de atención. El resultado es una reducción del uso de VRAM de hasta el 60% y una aceleración del entrenamiento de 1.7x-2.5x respecto a Hugging Face PEFT+TRL en QLoRA, verificada de forma independiente por múltiples equipos. La API de entrada es `FastLanguageModel.from_pretrained(model_name, max_seq_length=4096, load_in_4bit=True)`, que carga el modelo con las optimizaciones de Unsloth activas automáticamente y retorna el modelo y el tokenizador listos para aplicar `get_peft_model()`. Unsloth soporta Llama, Mistral, Gemma, Phi, Qwen y otros modelos principales, y tiene un feature especialmente valioso para el flujo de trabajo de despliegue: la exportación directa a GGUF con distintas variantes de cuantización (`model.save_pretrained_gguf("ruta", tokenizer, quantization_method="q4_k_m")`) sin necesidad de pasos de conversión adicionales.

Axolotl es la herramienta de mayor flexibilidad para equipos técnicos con necesidades avanzadas. El punto central de Axolotl es el archivo YAML de configuración que controla todos los aspectos del entrenamiento sin escribir código Python: el modelo base, el dataset y su formato, la estrategia de fine-tuning (full, LoRA, QLoRA, GaLore), el scheduler de learning rate (cosine, constant, warmup), el número de epochs, el gradient checkpointing, la evaluación periódica y el push automático al Hugging Face Hub al finalizar. La configuración de un fine-tuning QLoRA completo cabe en un YAML de 30-40 líneas; el comando de inicio es `accelerate launch -m axolotl.cli.train config.yaml`. Axolotl soporta DeepSpeed ZeRO stage 2 y 3 para entrenamiento multi-GPU con distribución automática del estado del

optimizador y los gradientes, y tiene integración con Weights & Biases para tracking de métricas de entrenamiento.

LLaMA-Factory está optimizado para la accesibilidad: su interfaz web LLaMA Board permite configurar y lanzar experimentos de fine-tuning a través de un formulario visual, sin línea de comandos ni código Python. Para equipos con experiencia en dominios de negocio pero menor profundidad técnica en ML, LLaMA-Factory puede ser la diferencia entre "necesitamos un ML engineer especializado" y "nuestro data scientist puede hacer el fine-tuning". Soporta SFT, RLHF, DPO y ORPO en modo gráfico, con importación de datasets en JSON, CSV y Hugging Face datasets con transformación automática al formato de chat.

TRL (Transformer Reinforcement Learning) de Hugging Face es el framework con mayor superficie de funcionalidades en el espacio de alignment. El `SFTTrainer` cubre el fine-tuning supervisado estándar con soporte para packing de secuencias y `DataCollatorForCompletionOnlyLM` para loss masking automático. Pero el diferenciador de TRL es su soporte para técnicas de alignment más avanzadas: `DPOTrainer` implementa Direct Preference Optimization sobre pares de respuestas `chosen` / `rejected`, permitiendo que el modelo aprenda preferencias humanas —o preferencias de un LLM juez— sin el ciclo completo de RLHF con PPO. DPO es especialmente valioso para mejorar el instruction following, reducir respuestas no deseadas y ajustar el tono y estilo del modelo después del SFT inicial. La configuración básica es `DPOTrainer(model, ref_model, args, tokenizer, train_dataset)` con un `beta=0.1` que controla la magnitud de la penalización por divergencia respecto al modelo de referencia. `ORPOTrainer` combina el SFT loss y el odds ratio preference loss en un único paso de entrenamiento, eliminando la necesidad de un modelo de referencia separado.

Características técnicas de cada herramienta

- **Unsloth:** kernels Triton para RoPE, cross-entropy y attention; 1.7-2.5x más rápido que PEFT+TRL en QLoRA; exportación directa a GGUF; API:
`FastLanguageModel.from_pretrained()`.
- **Axolotl:** configuración completa en YAML; soporte multi-GPU con DeepSpeed ZeRO 2/3; integración con Weights & Biases; inicio: `accelerate launch -m axolotl.cli.train config.yml`.
- **LLaMA-Factory:** interfaz web (LLaMA Board) sin código Python; soporte para SFT, DPO, RLHF en modo gráfico; para equipos sin experiencia profunda en ML.

- **TRL SFTTrainer:** entry point simple con soporte para packing, completion-only loss; `DPOTrainer` para preference fine-tuning con pares chosen/rejected; `ORPOTrainer` sin modelo de referencia.
- **Evaluación durante entrenamiento:** todas las herramientas soportan evaluation periódica en split de validación; integrar el golden dataset del Capítulo 1 como criterio de parada temprana.

***Nota del Arquitecto:** Para proyectos de producción con restricciones de tiempo y hardware, mi stack preferido es Unsloth para el loop de experimentación (más rápido, menor VRAM) y TRL con DPOTrainer para el refinamiento final del modelo si el SFT no alcanza el umbral de calidad deseado en instruction following. DPO sobre 500-1.000 pares de preferencias puede mejorar el comportamiento del modelo en casos problemáticos donde el SFT puro no llega al umbral de aceptación.*

Las herramientas de fine-tuning completan el stack técnico para producir modelos especializados. La sección de cierre sintetiza el capítulo y establece el fine-tuning eficiente como un elemento de la arquitectura de productos de IA, no como un experimento aislado.

Módulo 8 – Capítulo 06 – Sección 06

Cierre: el fine-tuning eficiente democratiza la especialización de modelos

La combinación de QLoRA, Unsloth y herramientas como Axolotl o LLaMA-Factory ha reducido el costo de producir un modelo especializado de decenas de miles de dólares en GPU time a menos de 50-200 dólares para modelos de 7B, con tiempos de entrenamiento de 2-8 horas en hardware de consumo o instancias cloud spot. Esta democratización tiene consecuencias arquitectónicas que van más allá de la eficiencia: transforma el tipo de decisiones que los equipos de ingeniería pueden tomar. En lugar de depender de un único modelo general que debe funcionar razonablemente bien en todos los casos de uso del producto, la arquitectura óptima para productos maduros frecuentemente involucra varios modelos pequeños especializados para distintas tareas, cada uno fine-tuneado en sus datos específicos y servido en vLLM con hot-swapping de adaptadores LoRA o como modelos fusionados independientes.

El ciclo de vida del fine-tuning se ha comprimido al punto donde la iteración es viable como práctica de ingeniería regular. El ciclo típico es: preparar o actualizar el dataset (horas a días), entrenar con Unsloth en una GPU de consumo o instancia spot (2-8 horas), evaluar en el golden dataset (minutos con lm-evaluation-harness), analizar los casos de fallo, actualizar el dataset con los edge cases descubiertos, y repetir. Este ciclo iterativo, impensable hace tres años cuando el fine-tuning requería clusters completos y semanas de tiempo de máquina, permite que los equipos de ingeniería desarrollen intuición empírica sobre qué funciona en sus datos específicos: cuánto dataset es suficiente, qué rank de LoRA captura el dominio, qué patrón de datos mejora los casos problemáticos.

Los adaptadores LoRA que resultan del fine-tuning son artefactos de primera clase en el sistema de despliegue, no archivos intermedios descartables. El adapter de 50-300 MB se versiona en Hugging Face Hub con el SHA del modelo base, el SHA del dataset de entrenamiento y las métricas de evaluación en el golden set como metadata. En vLLM con `--enable-lora --max-loras 4 --lora-modules nombre_adaptador=/ruta/al/adaptador`, múltiples adaptadores especializados pueden servirse simultáneamente sobre el mismo

modelo base sin recargar los pesos base entre switches: el sistema puede responder en el mismo servidor a una petición del adaptador de soporte técnico y la siguiente con el adaptador de generación de código, con el overhead de cambio de adaptador reducido a milisegundos.

Un aspecto que este capítulo no puede completar sin mencionar es el puente hacia el governance: los modelos fine-tuneados sobre datos propietarios son artefactos de alto valor que requieren protección. Los pesos ajustados con datos sensibles de la empresa pueden ser objeto de ataques de extracción de memorización; el pipeline de entrenamiento puede ser comprometido si el dataset de entrenamiento no está protegido; los modelos fine-tuneados que han removido safety training son más vulnerables a jailbreaking. El Capítulo 9 de governance y el Módulo 9 de seguridad abordan estas superficies de ataque con detalle.

Idea central

El fine-tuning eficiente con LoRA y QLoRA no es solo una técnica de optimización de recursos: es la base de una arquitectura de modelos especializados que supera en calidad y costo al paradigma de un único modelo general de gran tamaño para todas las tareas.

"Data is the new oil, but raw data is like crude oil: it has to be refined before it can power anything." — Clive Humby, matemático y pionero del marketing basado en datos, recordando que en fine-tuning de LLMs, la preparación del dataset de calidad es el trabajo más impactante y más frecuentemente subestimado del proceso.